Residual Frontier Refinement: Exact Shortest-Path Propagation with Adaptive Partial Ordering

Morgan Petrik
Militant.AI
mpetrik@militant.ai

Version 1.1.0
DOI (v1.0.0): 10.5281/zenodo.20329104

5 July 2026 (v1.0.0: 22 May 2026)

Abstract

Shortest-path solvers based on global priority queues impose a total order on the active frontier. This ordering is sufficient for exact propagation, but it is often stronger than necessary on structured graph and spatial routing problems. Residual Frontier Refinement (RFR) is an exact single-source shortest-path method that replaces global heap ordering with adaptive partial ordering over distance bands. A band is processed as a batch only when conservative safety checks prove that no unprocessed candidate can invalidate the propagation order. Ambiguous bands are split or resolved by a local exact fallback.

This whitepaper formalises the RFR algorithm, describes its safety invariant, defines its work model, and reports empirical observations from generated graph families and topographic routing demos. The current implementation validates exactness against Dijkstra across tested graph families. It also demonstrates that structured frontiers can avoid global ordering operations and expose useful diagnostic information about frontier ambiguity. Version 1.1 adds an invariant-band execution model for cost fields: when the band width does not exceed the minimum possible step cost, every band is unconditionally safe, eliminating runtime safety checks entirely; settling and relaxing whole bands as vectorized array operations then converts the proven independence into wall-clock performance. The invariant-band solver computes exact Dijkstra-identical distance fields 1.7×–5.5× faster than a hand-written array-based grid Dijkstra at 512^2–2048^2 points, with the advantage growing with scale, reproduced across two independent environments. The general-graph implementation remains slower than Dijkstra on pre-built graphs, and single-query workloads still favour A*; these limits are stated explicitly.

1 Contributions

This whitepaper makes four contributions:

1. It defines Residual Frontier Refinement as an exact shortest-path propagation method based on adaptive partial ordering.
2. It gives the conservative safety condition used to decide when a frontier band may be batch-processed.
3. It separates global heap-ordering work from local residual-resolution work, giving a clearer account of what the algorithm changes.

4. It reports the empirical and operational status: exactness is validated across tested graph families, work-shape benefits are visible, and wall-clock superiority is established for the invariant-band field solver but not for the general-graph implementation.

5. (v1.1) It introduces the invariant-band corollary — bands no wider than the minimum edge weight are unconditionally safe — and demonstrates that executing such bands as vectorized batches yields measured wall-clock superiority over grid-native DIJKSTRA baselines on cost fields (1.7×–5.5× at 512^2–2048^2 points, Section 8.5).

2 Problem Statement

Let $G = (V, E, w)$ be a directed graph with non-negative edge weights $w(u, v) \geq 0$, source $s \in V$, and shortest-path distances

$$d^\star(v) = \min_{p: s \rightsquigarrow v} \sum_{(u,z) \in p} w(u, z).$$

The single-source shortest-path problem computes $d^\star(v)$ for all reachable vertices and may also record predecessors for path reconstruction.

DIJKSTRA's algorithm [1] solves this problem by repeatedly extracting the globally minimum unsettled frontier element. The global minimum rule is exact, but it enforces a total order over the frontier. On structured graphs, many frontier vertices may be provably safe to process together, or safe to resolve using only local ordering, because their relative order cannot change the final distances. The core RFR hypothesis is:

> Shortest-path propagation requires enough ordering to preserve correctness, not necessarily complete global ordering at every step.

RFR tests this hypothesis by replacing the global heap with distance bands, conservative residual checks, splitting, and local fallback.

3 Algorithm Overview

3.1 Residual Frontier

RFR maintains a frontier $\mathcal{F}$ partitioned into bands. Each band $\mathcal{B}$ has:

- a lower and upper distance bound $[\ell(\mathcal{B}), u(\mathcal{B}))$,
- a set of active members $\{(v, \hat{d}(v))\}$,
- update counts and refinement depth,
- residual diagnostics measuring internal width, volatility, cross-edge risk, and degree risk.

The implementation records the following residual components for a band:

$$R(\mathcal{B}) = R_{\text{width}}(\mathcal{B}) + R_{\text{volatility}}(\mathcal{B}) + R_{\text{cross}}(\mathcal{B}) + R_{\text{degree}}(\mathcal{B}).$$

These residuals are not trusted as correctness criteria by themselves. They guide splitting and local fallback. Exactness depends on the conservative safety rules in Section 3.3.

3.2 High-Level Procedure

```
ResidualFrontierSSSP(G, s):
    dist[v] <- infinity for all v
    dist[s] <- 0
    frontier.insert_or_update(s, 0)

    while frontier is not empty:
        B <- lowest non-empty distance band

        if Safe(B, frontier, dist, G):
            batch <- all members of B sorted locally by tentative distance
            remove B from frontier
        else:
            residual <- measure residual pressure in B
            if residual exceeds threshold and B can be refined:
                split B into smaller bands
                continue
            else:
                batch <- one local exact minimum from B

        for (u, du) in batch:
            if u is stale or settled:
                continue
            settle u
            for each outgoing edge (u, v, w):
                candidate <- du + w
                if candidate < dist[v]:
                    dist[v] <- candidate
                    predecessor[v] <- u
                    if v is unsettled:
                        frontier.insert_or_update(v, candidate)
```

3.3 Safe Batch Extraction

Let $\mathcal{B}$ be the lowest non-empty band in the frontier. Let

$$d_{\max}(\mathcal{B}) = \max_{v \in \mathcal{B}} \hat{d}(v)$$

and let

$$d_{\min}(\mathcal{F} \setminus \mathcal{B}) = \min_{v \in \mathcal{F} \setminus \mathcal{B}} \hat{d}(v),$$

with $d_{\min} = \infty$ if no outside frontier elements exist.

RFR first requires:

$$d_{\max}(\mathcal{B}) \leq d_{\min}(\mathcal{F} \setminus \mathcal{B}).$$

This ensures no already-known outside frontier element precedes the band.

The implementation also checks whether relaxing a member of $\mathcal{B}$ could create an outside candidate that should precede remaining band members. For each $u \in \mathcal{B}$ and outgoing edge (u, v) with

$v \notin \mathcal{B}$, the safety check rejects the batch if:

$$\hat{d}(u) + w(u, v) < d_{\max}(\mathcal{B}).$$

If this condition occurs, processing the whole band could skip an outside candidate that should be interleaved before later band members. The band is then unsafe and must be split or resolved locally.

3.4 Unsafe Bands

Unsafe bands represent frontier ambiguity. RFR responds in two ways:

1. **Split:** If the band has multiple members and has not exceeded the maximum refinement depth, it is split into narrower bands.
2. **Local exact fallback:** If splitting is no longer useful or allowed, the algorithm extracts the exact local minimum from the band.

The local fallback is not a correctness failure. It is a controlled reversion to exact ordering where partial ordering is insufficient.

3.5 Compositional Design

RFR is intentionally decomposed into small, independently inspectable parts:

- **Safety predicates** decide whether a frontier band can be processed without violating exactness.
- **Band transformations** insert, update, split, and remove frontier regions without settling vertices.
- **Residual measurements** describe ambiguity pressure but do not replace the safety predicates.
- **Local fallback** provides controlled recovery to exact local ordering when partial ordering is insufficient.

This separation is operationally important. A conventional global priority queue hides most ordering pressure inside heap operations. RFR exposes that pressure as data: a band was safe, unsafe, split, locally resolved, or accepted as a batch. The result is not only a distance field, but also a diagnostic trace of where the problem required precision.

4 Correctness Argument

4.1 Invariant

RFR maintains the standard label-setting invariant [2]:

> When a vertex u is settled, its tentative distance $\hat{d}(u)$ equals the true shortest-path distance $d^\star(u)$.

Dijkstra enforces this invariant by extracting the global minimum unsettled label. RFR enforces it by proving that either a full band is safe, or by falling back to an exact local minimum within an unsafe band.

4.2 Safe Band Lemma

Lemma. If a band $\mathcal{B}$ passes the two safety checks in Section 3.3, then settling the members of $\mathcal{B}$ in nondecreasing tentative-distance order preserves the label-setting invariant.

Sketch. The first safety check ensures that no known outside frontier vertex has a smaller tentative distance than any member of $\mathcal{B}$. The second safety check ensures that relaxing any band member cannot create an outside candidate whose distance is smaller than the maximum remaining distance within the band. Therefore, no outside candidate needs to be interleaved before the batch completes. Sorting the batch internally by tentative distance gives the same order that a global priority queue would observe over this locally isolated subset. Thus each settled vertex has the globally minimal necessary label at its settlement point.

4.3 Unsafe Band Lemma

Lemma. If a band is unsafe, splitting or extracting a local exact minimum preserves the possibility of exact label-setting propagation.

Sketch. Splitting does not settle any vertex; it only refines the frontier partition. It therefore cannot invalidate distances. Local exact fallback extracts the minimum label inside the current lowest band. Since the band is the lowest non-empty band by lower bound, and exact local extraction is used when batch safety is unavailable, this step conservatively approximates DIJKSTRA's global-minimum behaviour within the ambiguous region. Stale entries are ignored, and relaxations are accepted only when they improve labels.

4.4 Theorem

Theorem. For graphs with non-negative edge weights, RFR computes the same shortest-path distances as DIJKSTRA, assuming the safety checks and stale-entry rules described above.

Sketch. By induction over settled vertices. The source is settled with distance zero. At each step, either a safe band is processed under the Safe Band Lemma or an unsafe band is refined/resolved under the Unsafe Band Lemma. Relaxation is identical to DIJKSTRA once a vertex is selected. Since no settled vertex violates the label-setting invariant, all reachable final distances match $d^\star$. Unreachable vertices remain at infinity.

4.5 Invariant Band Corollary

Corollary. Let $w_{\min} = \min_{(u,v)\in E} w(u,v) > 0$. If every band satisfies $u(\mathcal{B}) - \ell(\mathcal{B}) \le w_{\min}$, and the lowest non-empty band is always processed first, then every band passes both safety checks of Section 3.3 unconditionally.

Proof. Because bands partition distances by fixed key and the lowest non-empty band is processed first, every outside frontier member lies in a higher band, so $d_{\min}(\mathcal{F} \setminus \mathcal{B}) \ge u(\mathcal{B}) > d_{\max}(\mathcal{B})$, satisfying the first check. For the second check, any candidate created by relaxing $u \in \mathcal{B}$ has value $\hat{d}(u) + w(u,v) \ge \ell(\mathcal{B}) + w_{\min} \ge \ell(\mathcal{B}) + \big(u(\mathcal{B}) - \ell(\mathcal{B})\big) = u(\mathcal{B}) > d_{\max}(\mathcal{B})$. No relaxation can produce a candidate that precedes any remaining band member. □

The consequence is architectural: for problem classes where $w_{\min}$ is known *statically* — in particular cost fields, where the minimum differential is the grid resolution plus the minimum cell increment — band safety requires no runtime inspection at all. The safety proof is purchased once, from the invariant, instead of per-band at retail. Section 7 develops this into an execution model. This corollary also locates RFR precisely in the bucket lineage: it is Dial's condition [3] generalised to real weights, and delta-stepping [4] with $\Delta = w_{\min}$, where minimum-weight edges are light,

heavier edges cross bucket boundaries in one outer pass, and bucket width coincides with the static invariant.

5 Work Model

RFR separates two forms of work that are conflated in ordinary priority-queue shortest-path implementations.

5.1 Dijkstra Work

The baseline global ordering work is:

$$W_{\text{global}} = H_{\text{push}} + H_{\text{pop}},$$

where H_{push} and H_{pop} are heap insertions and heap removals. The implementation records these as `global_heap_pushes` and `global_heap_pops`.

5.2 RFR Work

RFR records local resolution work:

$$W_{\text{local}} = S_{\text{band}} + R_{\text{local}} + F_{\text{heap}} + I_{\text{heap}},$$

where S_{band} is band scan work, R_{local} is local refinement count, F_{heap} is local heap fallback count, and I_{heap} is the number of items resolved by local fallback.

The implementation also records safe batches, band splits, frontier peak size, stale entries, relaxations, distance updates, and batch vertices.

5.3 Resolution Efficiency

The project uses a diagnostic ratio:

$$E = \frac{W_{\text{global avoided}}}{W_{\text{local}}}.$$

Values above one indicate that RFR avoided more global heap-ordering operations than it added in local resolution work. This is a work-shape metric, not a direct wall-clock claim.

6 Adaptive Policies

6.1 Static Graph Classification

The implementation computes graph features:

- vertex and edge counts,
- density,
- average degree,
- clustering,

- edge-weight mean and variance.

These features select an initial frontier policy:

Graph signal	Policy response
Road-like geometry	broad directional bands, larger scan limit
Cluster structure	basin-first propagation, moderate bands
High ambiguity	narrow bands and local heap fallback
Low ambiguity	broad bands with moderate residual threshold
Medium ambiguity	split-oriented hybrid bands

6.2 Probe Revision

RFR v4 introduced a probe phase:

$$\text{graph features} \to \text{initial policy} \to \text{frontier probe} \to \text{revised policy}.$$

The probe observes split pressure, fallback rate, safe-batch ratio, residual size, cross-edge risk, and volatility. RFR v5 removes duplicate traversal by making the probe the opening segment of the exact run; the policy is revised in place and propagation continues.

6.3 Hybrid Indexed Frontier

RFR v6 adds a hybrid frontier:

$$\begin{cases} \text{indexed residual frontier}, & \text{structured policies}, \\ \text{split-capable v5 frontier}, & \text{ambiguous policies}. \end{cases}$$

The indexed frontier computes band placement directly from the priority and caches band extrema. It helps road-like cases but is not a universal improvement. Random and irregular graphs need split-capable behaviour, so the hybrid avoids applying the indexed structure where it is a poor fit.

7 Invariant-Band Execution on Cost Fields

The corollary of Section 4.5 removes the safety-checking cost, but proven band independence must still be *spent* on something. Sequential execution spends it on nothing: a batch processed one element at a time is indistinguishable from heap extraction. The v1.1 solver spends it on data parallelism.

7.1 Execution Model

For a $W \times H$ cost field with resolution r and cell costs $c \geq 0$, set the static band width $\delta = r + \min c$. Distances, settlement flags, and cell costs live in flat arrays indexed by $y \cdot W + x$. Nodes are filed into band buckets keyed by $\lfloor \hat{d}/\delta \rfloor$ at update time, with lazy deletion. Processing the lowest non-empty bucket settles all live members at once and relaxes each stencil direction as a single whole-array operation:

```
InvariantBandField(W, H, cost, r):
    delta <- r + min(cost)            # static safety invariant
    dist[*] <- infinity; dist[source] <- 0
    file source in bucket[0]
    k <- 0
    while buckets remain:
        B <- unique live members of bucket[k]   # lazy deletion
        settle all of B                          # unconditionally safe
        for each stencil direction d:            # 4 vectorized ops
            T <- neighbours of B in direction d
            cand <- dist[B] + step_cost[T]
            improved <- cand < dist[T]
            dist[T[improved]] <- cand[improved]
            file T[improved] in bucket[floor(cand/delta)]
        k <- k + 1
```

Exactness follows directly from the corollary; the implementation (about fifty lines of NumPy) validates distance-field equality against Dijkstra at every benchmarked size. Total element work is $O(N)$ with array-operation constants; ordering overhead is $O(d_{\max}/\delta)$ bucket steps, i.e. proportional to field diameter rather than to $N \log N$ heap traffic.

7.2 What Changed Relative to v1.0

The v1.0 spatial solver (Section 9) already avoided graph materialisation but executed sequentially with per-edge function dispatch, per-point tuple keys, and per-lookup layer dispatch. Those costs made it 5–7× slower than a plain array-based grid Dijkstra despite the sound architecture (Section 8.4). The v1.1 solver keeps the thesis and deletes the overhead: safety by invariant instead of inspection, batches spent as array parallelism instead of sequential loops, flat arrays instead of hashed points.

8 Empirical Evidence

8.1 Correctness Tests

The test suite validates equality with Dijkstra on hand-built graphs, generated road-like grids, clustered graphs, random graphs, irregular weighted graphs, unreachable vertices, and adaptive variants v3 through v6. The core tested condition is:

$$d_{\mathrm{RFR}}(v) = d_{\mathrm{Dijkstra}}(v)$$

for all reachable vertices in each test instance.

8.2 Benchmark Smoke Results

Early smoke benchmarks showed exactness and useful work decomposition. Table 1 summarises representative v4 results from generated graph families.

Family	Correct	Global order avoided	Safe batches	Band splits
Road-like grid	true	754	37	1
Clustered basins	true	256	17	0
Random dense-ish	true	584	22	16
Irregular weighted	true	498	12	10

Table 1: Representative feedback-driven RFR smoke results. Structured cases allow many safe batches; ambiguous cases require more splitting.

8.3 Operational Graph Results

Operational benchmarking compares Python wall-clock time against Dijkstra on pre-built graph families. The current Python implementation is slower than Dijkstra in all tested graph-family runs. Table 2 summarises median v6 hybrid observations.

Family	Time ratio	Efficiency	Safe batches
Road-like grid	4.85	1.89	37
Clustered basins	7.90	1.77	17
Random dense-ish	9.09	3.91	24
Irregular weighted	8.80	3.61	12

Table 2: Median operational results for the v6 hybrid frontier. All listed families were correct against Dijkstra. Exactness and work-shape improvements are validated; wall-clock superiority is not validated in this Python implementation.

The interpretation is deliberately conservative. RFR avoids global ordering operations and exposes frontier ambiguity, but Python-level frontier machinery, safe-band scans, and local sorting dominate runtime for the current implementation.

8.4 Grid-Native Baselines for the v1.0 Spatial Solver

Independent adversarial benchmarking (harness included in the artefacts) compared the v1.0 spatial solver against the baselines practitioners actually deploy on fields: a plain array-based grid Dijkstra that never materialises a graph, and A* with a Manhattan heuristic for single queries. All methods produced identical distances. Ratios are v1.0 spatial time over baseline time (medians; > 1 means the baseline is faster).

8.5 Invariant-Band Wall-Clock Results

The invariant-band solver of Section 7 was benchmarked on the same fields, seeds, and cost model (CPython 3.10, NumPy, one CPU core; medians; distance-field equality verified against Dijkstra at every size).

The results were independently reproduced on the author's hardware (Windows, CPython, NumPy 1.26; medians of three repeats; all correctness checks passed): 1.7×, 2.7×, and 3.9× over grid Dijkstra at 512^2, 1024^2, and 2048^2 weighted respectively, 5.5× at 1024^2 unweighted, with the same crossover below roughly 10^5 points. Across both environments the invariant-band solver is 10×–49× faster than the v1.0 spatial solver.

Grid	Weighted	vs. build+solve	vs. grid DIJKSTRA	vs. A* (single query)
32^2	no	0.71	5.11	3.79
96^2	no	0.60	5.64	3.58
96^2	yes	0.61	6.01	3.45
256^2	yes	0.55	5.98	3.18
512^2	yes	0.60	6.61	3.34

Table 3: v1.0 spatial solver against practitioner baselines. The advantage over graph-materialisation workflows is real and grows to 40–45% at scale (first column), but grid-native implementations that also skip materialisation are 5–7× faster than the v1.0 solver. The deficit is implementation overhead, not algorithmic: per-edge closure dispatch, per-lookup layer dispatch, and hashed point keys.

Grid	Weighted	Invariant-band (s)	Grid DIJKSTRA (s)	Speedup	vs. v1.0 spatial
96^2	no	0.0048	0.0043	0.88×	5.0×
96^2	yes	0.0065	0.0047	0.73×	4.3×
512^2	yes	0.0863	0.1626	1.88×	12.6×
1024^2	yes	0.2598	0.7228	2.78×	23.6×
1024^2	no	0.1155	0.5617	4.86×	44.8×
2048^2	yes	0.8586	3.2666	3.80×	—

Table 4: Invariant-band solver wall-clock results. Below roughly 10^5 points, per-band array-operation overhead dominates and the plain grid DIJKSTRA remains competitive; above it, the invariant-band solver wins and the advantage grows with scale, consistent with $O(N)$ element work against heap traffic. All results are exact.

This is the empirical form of the paper's thesis: ordering work that a static invariant proves unnecessary can be discarded, and when the resulting independence is executed as batch parallelism rather than sequentially, discarding it is not merely exact but faster.

8.6 Topographic Routing Case Study

The TopographicMaps simulator applies terrain Dijkstra and terrain RFR to Digital Elevation Model rasters. It compares target discovery work rather than full-map post-target exploration. In the current eight-sample public DEM set, the median target-work reduction is approximately 49.19%, with observed savings between 48.57% and 49.58%.

DEM	Dijkstra target work	RFR target work	Work saved
o41078a5	2386	1203	49.6%
o41078a1	2525	1286	49.1%
o41078a2	2475	1273	48.6%
o41078a3	2453	1249	49.1%
o41078a4	2367	1200	49.3%
o41078a6	2406	1213	49.6%
o41078a7	2424	1240	48.8%
i30dem	2048	1035	49.5%

Table 5: Topographic target-discovery work in the simulator. The metric compares global heap-order work for terrain DIJKSTRA against local band-resolution work for terrain RFR.

This case study supports the visual and diagnostic value of the method. It should not be read as a universal timing result because the simulator compares work units and because implementation details affect wall-clock performance.

9 Operational Applications and Implications

RFR is not only an alternative implementation of shortest paths. Its residual frontier model creates practical leverage in domains where the map is structured, where the frontier is interpretable, or where costs change over time. The most important applications are those where the graph is implicit or expensive to materialise, and where a solver must be integrated into a larger operational system rather than run as an isolated benchmark.

9.1 Direct Operation on Maps, Grids, and Rasters

Many real pathing domains begin as maps rather than abstract graphs: game tiles, navmesh regions, occupancy grids, terrain rasters, heatmaps, and GIS layers. A graph can be materialised from these structures, but doing so introduces construction cost, memory overhead, and another representation boundary. The spatial components of this project show the practical value of treating each point as a local evaluator:

$$value(p) = \min_{q \in N(p)} value(q) + \Delta(q, p, \text{layers}).$$

In that setting, RFR's band and residual model can operate over structured frontier regions rather than over an arbitrary heap of graph vertices. This does not remove the need for a

correctness-preserving ordering rule, but it aligns the ordering mechanism with the native structure of maps and rasters.

9.2 Games and Structured Level Pathfinding

Games commonly exploit structure through grids, navmeshes, hierarchical abstraction, and clustered regions. RFR is complementary to these practices. On a structured level, broad coherent bands may correspond to corridors, basins, rooms, road-like lanes, or terrain-cost contours. Safe-batch ratios, split pressure, and fallback rates can expose whether the level geometry is producing coherent pathing frontiers or chaotic ordering pressure.

That diagnostic value is useful during development. A designer or tools engineer can ask not only "what path was selected?" but also "where did the frontier become ambiguous?" and "which regions forced exact local ordering?" This makes RFR a potential development instrument even where another production solver remains the runtime default.

9.3 Dynamic Repathing

RFR is not yet an incremental shortest-path algorithm in the sense of D* Lite [5]. However, its architecture points naturally toward local repair. Dynamic map changes usually affect a region of the frontier, not the entire metric space. Because RFR already represents the frontier as bands with residual pressure, updates could invalidate or refine only the affected bands, then resume propagation from the disturbed region.

This is a future-work direction rather than a validated claim. The important architectural point is that RFR separates the frontier into inspectable, transformable units. That is more amenable to local repair than an opaque global heap whose ordering state is meaningful only as a single total order.

9.4 Topographic and Terrain-Aware Routing

Terrain routing for drones, outdoor robotics, and GIS-style planning often combines distance, slope, resistance, masks, risk, and domain-specific movement costs. These costs tend to be spatially coherent. The TopographicMaps simulator demonstrates that RFR can expose and exploit this coherence under a target-query work metric while preserving the same terrain-cost answer as Dijkstra.

The operational implication is not that the current Python implementation is a production robotics solver. It is that terrain-aware routing is a natural fit for partial ordering: large regions of the frontier may be safely accepted together, while steep slopes, barriers, and high-resistance zones create visible residual pressure.

10 Relationship to Existing Techniques

Dijkstra enforces exactness by total global ordering. Delta-stepping [4] groups tentative distances into buckets and can process light-edge relaxations in parallel, often trading strict ordering for controlled bucket processing. RFR is related in spirit but differs in two important ways:

1. RFR treats bucket or band processing as a safety problem, not only as a fixed-width scheduling choice.

2. RFR records residual pressure and can split or locally resolve an unsafe band rather than relying solely on a static bucket width.

The practical comparison target is therefore not only DIJKSTRA, but also bucketed, radix, delta-stepping, and domain-specialised shortest-path implementations.

The invariant-band solver of Section 7 makes the lineage exact: with band width fixed at $w_{\min}$ it coincides with Dial's bucket condition [3] generalised to real weights, and with delta-stepping at $\Delta = w_{\min}$, where minimum-weight edges are light and each bucket can be settled before heavier relaxations propagate outward. What RFR adds to that lineage is (i) the general safety machinery for bands *wider* than $w_{\min}$, where unconditional safety fails and inspection, splitting, or fallback is required, and (ii) the residual diagnostic account of where ordering pressure lives.

The thesis itself — that Dijkstra's total ordering is not required for exact SSSP — has since received strong independent corroboration: Duan, Mao, Mao, Shu, and Yin give a deterministic $O(m \log^{2/3} n)$ algorithm for directed SSSP with real non-negative weights, the first to break the sorting barrier on sparse graphs [6]. Their result is theoretical and unrelated in mechanism, but it establishes at the level of asymptotic proof what this project pursues operationally: global frontier ordering is an implementation strategy, not a correctness requirement.

10.1 Incremental Search

Incremental methods such as D* Lite reuse previous search information when edge costs change. RFR does not currently implement D* Lite's incremental consistency machinery. Its relevance is architectural: residual bands identify where ordering uncertainty lives, which could provide a natural repair unit for future dynamic variants. A dynamic RFR extension would need to define invalidation, band repair, and predecessor repair rules explicitly before claiming equivalence to established incremental algorithms.

10.2 Hierarchical Pathfinding

Hierarchical methods such as HPA* [7] reduce search cost by abstracting a map into clusters and solving at multiple resolutions. RFR addresses a different layer of the problem. It does not replace hierarchy; it changes how a frontier is ordered inside a level of representation. The two approaches are compatible: hierarchy can reduce the search domain, while RFR can resolve the frontier within a cluster, corridor, or refined local region.

This distinction is important for games. A production game pathing stack may still use navmeshes, HPA*, flow fields, or cached tactical maps. RFR's contribution is a safety-checked partial ordering mechanism that can sit inside such systems, especially where residual diagnostics are valuable.

11 Limitations and Non-Claims

The current state of RFR supports several claims and explicitly rejects others.

11.1 Validated Claims

- RFR preserves exact shortest-path distances on tested non-negative graph families.
- Structured frontiers can be processed in safe batches.
- The algorithm exposes where ordering pressure occurs through residual history, splits, fallback, and safe-batch ratios.

- Adaptive policy revision improves behaviour when static graph features misclassify frontier ambiguity.
- The topographic simulator shows a clear target-work reduction under the current work metric.
- (v1.1) The invariant-band field solver computes exact distance fields 1.7×–5.5× faster than hand-written grid DIJKSTRA at 512^2–2048^2 points (Section 8.5).
- The architecture creates practical integration points for map-native routing, diagnostics, and future local repair.

11.2 Non-Claims

- The general-graph Python implementation is not faster than DIJKSTRA on pre-built graph benchmarks.
- The v1.0 reference spatial solver is 5–7× slower than a plain array-based grid DIJKSTRA; its measured advantage applies to graph-materialisation workflows only (Table 3).
- Invariant-band wall-clock gains require full-field computation on fields of roughly 10^5 points or more; single-query workloads still favour A*.
- RFR is not expected to outperform DIJKSTRA on every graph family.
- Resolution efficiency is not identical to wall-clock speed.
- The indexed frontier is not universally beneficial; it is useful only under policies that predict coherent structure.
- The present implementation is not yet an incremental replanning algorithm and should not be represented as a replacement for D* Lite.
- The present work does not claim to solve multi-agent pathfinding or conflict resolution.

11.3 Implementation Bottlenecks

Current bottlenecks include:

- Python-level band management overhead,
- safe-band scans,
- local sorting and local heap fallback,
- insufficiently specialised data structures for ambiguous frontiers,
- absence of GPU and compiled-core implementations of the invariant-band solver,
- absence of validated incremental repair machinery for dynamic maps.

12 Future Work

Promising next steps are:

1. implement a low-overhead C++ or Rust frontier core to test wall-clock performance without Python interpreter overhead,
2. implement a split-capable indexed frontier,
3. add a cluster-specific basin frontier,
4. reduce safe-band sorting in coherent road-like frontiers,
5. extend the invariant-band execution model with GPU band kernels and a compiled (Rust or C++) core,
6. profile invariant-band scaling beyond 10^7 points and on full-resolution DEM rasters,
7. distinguish target-query work from full-field computation in all demos and reports,
8. develop an incremental RFR variant for dynamic costs and local repair,
9. integrate RFR within hierarchical game pathing stacks such as clustered grids or navmesh regions,
10. fold the invariant-band solver into the packaged library behind the layered-field API, keeping the v1.0 solver as the exactness oracle.

13 Conclusion

Residual Frontier Refinement reframes exact shortest-path propagation as a problem of resolving only the frontier ordering uncertainty that can affect correctness. It replaces unconditional global heap ordering with conservative band safety checks, residual-guided splitting, and local exact fallback. The current implementation validates exactness, provides a useful diagnostic account of frontier ambiguity, and shows clear work-shape benefits on structured and spatial cases. Version 1.1 converts the thesis into a measured result: on cost fields, band independence proven from a static invariant and executed as vectorized batches computes exact distance fields 1.7×–5.5× faster than hand-written grid Dijkstra at 512^2–2048^2 points, while the general-graph implementation remains slower than Dijkstra and single queries still favour A*. The defensible conclusion is now stronger than in v1.0: global total ordering is not inherent to shortest-path correctness — it is one implementation strategy — and where the required independence can be proven from static structure and spent as parallelism, discarding it is not merely exact but faster.

Acknowledgements

The grid-native baseline benchmarks, the invariant-band corollary formulation, and the reference NumPy implementation of the invariant-band solver in this revision originated in an adversarial review session conducted with Claude (Anthropic). All figures are reproducible from the benchmark harness included in the release artefacts.

References

[1] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, third edition, 2009.

[3] R. B. Dial. Algorithm 360: Shortest-path forest with topological ordering. *Communications of the ACM*, 12(11):632–633, 1969. https://doi.org/10.1145/363269.363610

[4] U. Meyer and P. Sanders. Delta-stepping: A parallelizable shortest path algorithm. *Journal of Algorithms*, 49(1):114–152, 2003. https://doi.org/10.1016/S0196-6774(03)00076-2

[5] S. Koenig and M. Likhachev. D* Lite. In *Proceedings of the 18th National Conference on Artificial Intelligence (AAAI)*, pages 476–483, 2002.

[6] R. Duan, J. Mao, X. Mao, X. Shu, and L. Yin. Breaking the sorting barrier for directed single-source shortest paths. arXiv:2504.17033, 2025. https://arxiv.org/abs/2504.17033

[7] A. Botea, M. Müller, and J. Schaeffer. Near optimal hierarchical path-finding. *Journal of Game Development*, 1(1):7–28, 2004.